

Hotel Room Allotment System

Project Report

Student Name	Shivam Kumar
Program	B.Tech Computer Science & Engineering
Institution	VIT Bhopal University
Subject / Course	B.Tech CSE Core
Submission Type	Flipped Course Evaluation — Build Your Own Project
Repository	https://github.com/shivam6205/Hotel-management.git

1. Introduction

Hotels and small guesthouses that rely on manual, paper-based, or spreadsheet-based booking records commonly run into avoidable problems: rooms get double-booked, front-desk staff lose track of which rooms are occupied, and simple tasks like finding a guest's booking or calculating a checkout bill take longer than they should.

This project implements a **Hotel Room Allotment System** as a command-line Python application. It models a small hotel's rooms and bookings using core Python data structures — lists and dictionaries — and provides a menu-driven interface through which front-desk staff can view rooms, book a room for a guest, search and manage existing bookings, process checkouts with automatic billing, and view an overall occupancy summary.

The project was built as part of a first-semester Computer Science course to demonstrate practical use of variables, data types, conditionals, loops, functions, lists, dictionaries, string formatting, and basic input validation — all within a single, coherent, working application rather than isolated exercises.

2. Problem Statement

Small hotels managing bookings manually face three recurring issues:

1. **Double-booking** — the same room can be assigned to two guests if staff do not carefully cross-check a register.
2. **Slow lookups** — finding a specific guest's booking, or checking a room's current status, requires manually scanning paper records.
3. **Manual billing errors** — calculating a checkout bill (nights stayed × room rate) by hand is error-prone.

The system addresses these by keeping a single, consistent, in-memory record of every room's status and every active booking, and by enforcing rules (such as blocking a booking on an already-occupied room) directly in code rather than relying on staff vigilance.

3. Functional Requirements

The system is organized into three major functional modules:

Module 1 — Room & Availability Management

- Display all rooms with type, price, and current status
- Filter and display only currently available rooms

- Track room status (Available / Booked) in real time

Module 2 — Booking Management

- Accept guest details: name, age, phone number, number of guests
- Validate room existence and availability before confirming a booking
- Block double-booking of an already-occupied room
- Generate a unique Booking ID (B001 , B002 , ...) per booking
- Display a booking confirmation summary

Module 3 — Search, Cancellation & Checkout

- Retrieve booking details by Booking ID
- Search bookings by guest name
- Cancel a booking and revert the room to Available
- Process checkout: total = nights × room price, then free the room
- Generate a hotel-wide occupancy summary by status and room type

Input / Output Structure

Action	Input	Output
View rooms	Menu choice	Table of all rooms with status
View available rooms	Menu choice	List of currently free rooms
Book a room	Room number, guest details, Y/N confirmation	Booking confirmation with Booking ID
View booking details	Booking ID	Full booking record
Search guest	Guest name	Matching booking details
Cancel booking	Booking ID, Y/N confirmation	Cancellation confirmation, room freed
Checkout	Booking ID, number of nights	Total bill, checkout confirmation
Hotel summary	Menu choice	Occupancy statistics

User Workflow

The application runs as a continuous loop: the main menu is displayed, the user selects an option (1–9), the corresponding module executes, a result is shown, and control returns to the menu until the user selects Exit. See Section 7.2 for the full workflow diagram.

4. Non-Functional Requirements

Requirement	How it is addressed
Usability	A clear, numbered menu (1–9) with descriptive labels allows a non-technical user to operate the system without instructions. Y/N confirmation prompts precede irreversible actions such as booking and cancellation.
Reliability	Room status and booking records stay consistent — a room is only marked <code>Booked</code> after a successful booking and is reliably reverted to <code>Available</code> on cancellation or checkout.
Error Handling / Robustness	Input is validated at each step using <code>try/except ValueError</code> for numeric fields; invalid menu choices and non-existent rooms/bookings are handled with clear messages instead of crashing.
Maintainability	The code is organized into small, single-purpose functions (<code>book_room()</code> , <code>cancel_booking()</code> , <code>check_out()</code> , etc.), making each feature independently modifiable.
Resource Efficiency	The system uses lightweight in-memory lists and dictionaries with no external database or network dependency, so it runs instantly on any machine with a standard Python interpreter.

5. System Architecture

The system follows a simple three-layer structure appropriate for a single-file, first-semester Python project: a presentation layer (the menu loop), a business logic layer (the feature functions), and a data layer (in-memory lists of dictionaries).

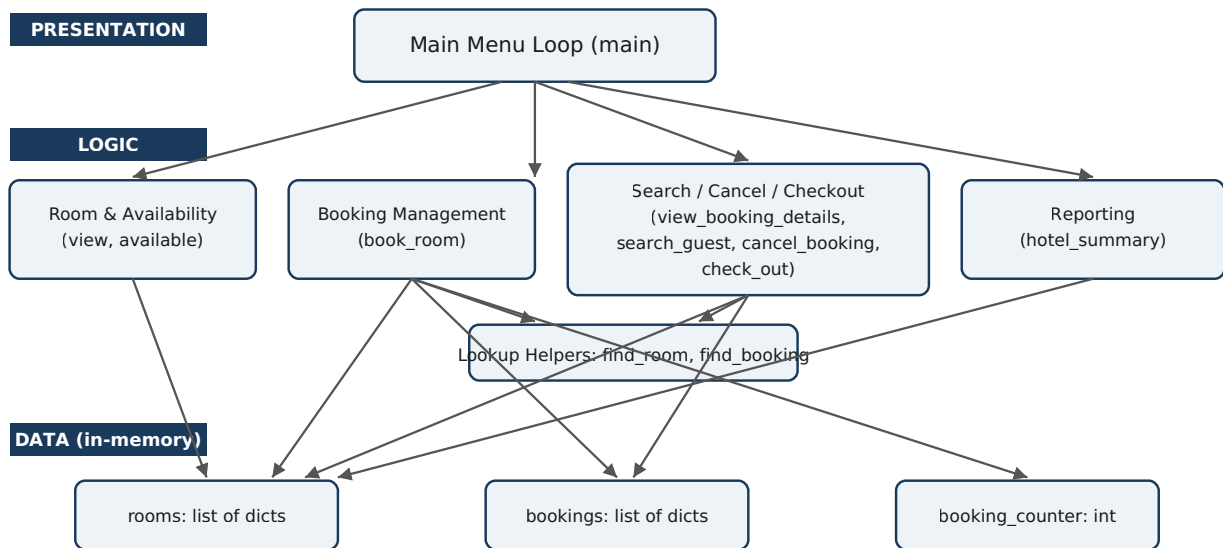


Figure 1: System Architecture Diagram

7. Design Diagrams

7.1 Use Case Diagram

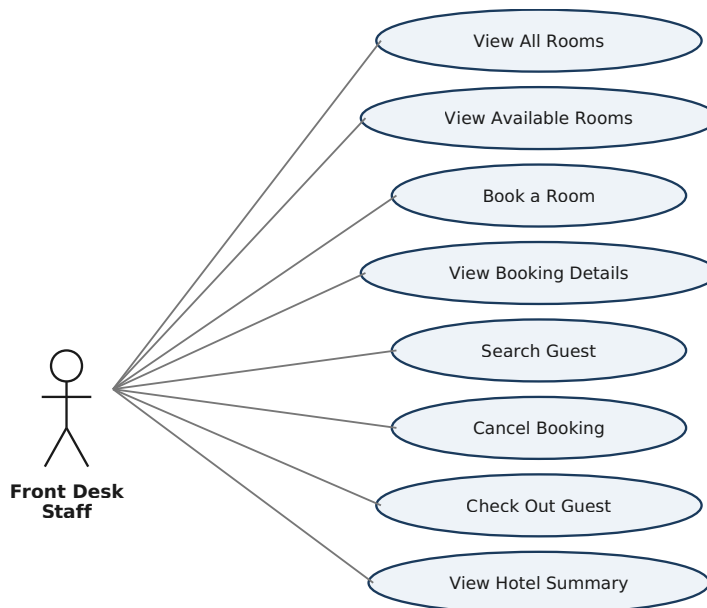


Figure 2: Use Case Diagram — Front Desk Staff

7.2 Workflow Diagram

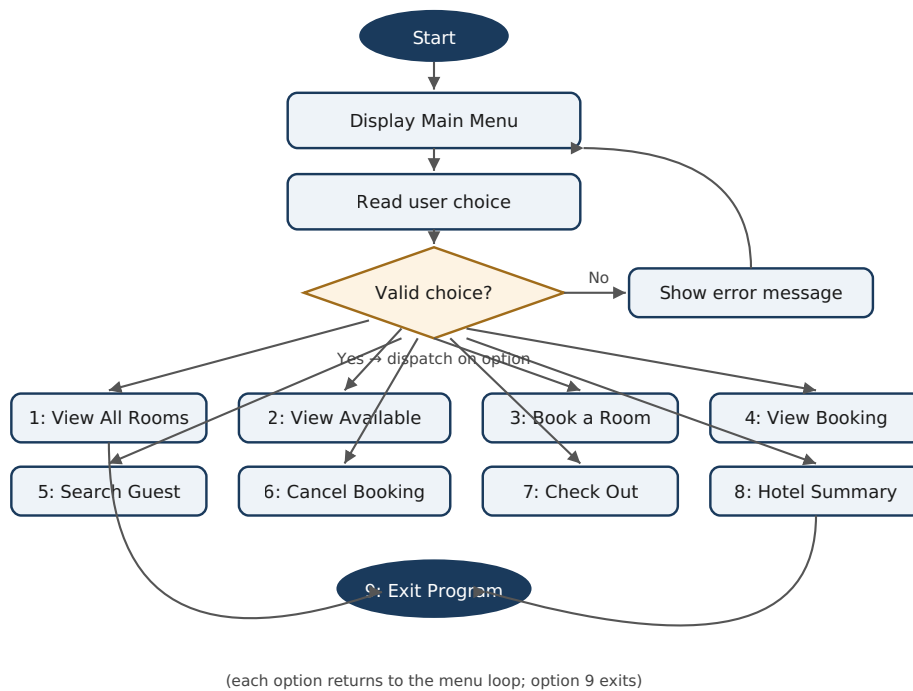


Figure 3: Main Menu Workflow

7.3 Class / Component Diagram

The implementation is function-based rather than class-based, consistent with first-semester scope. This diagram represents the system as functional components operating on two record types.

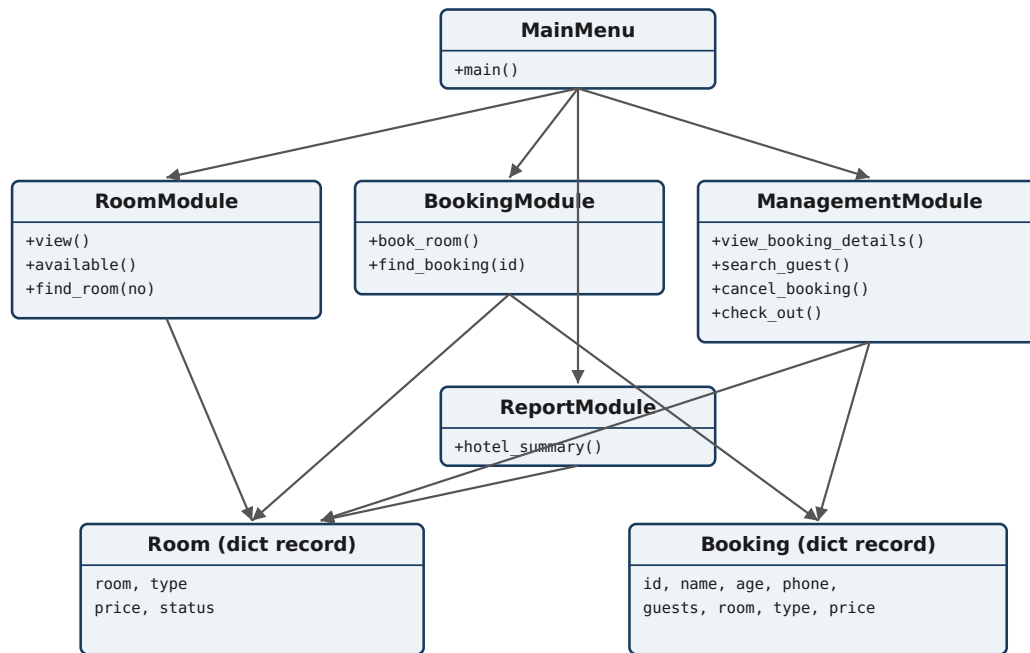


Figure 4: Component Diagram

7.4 Sequence Diagram — Booking a Room

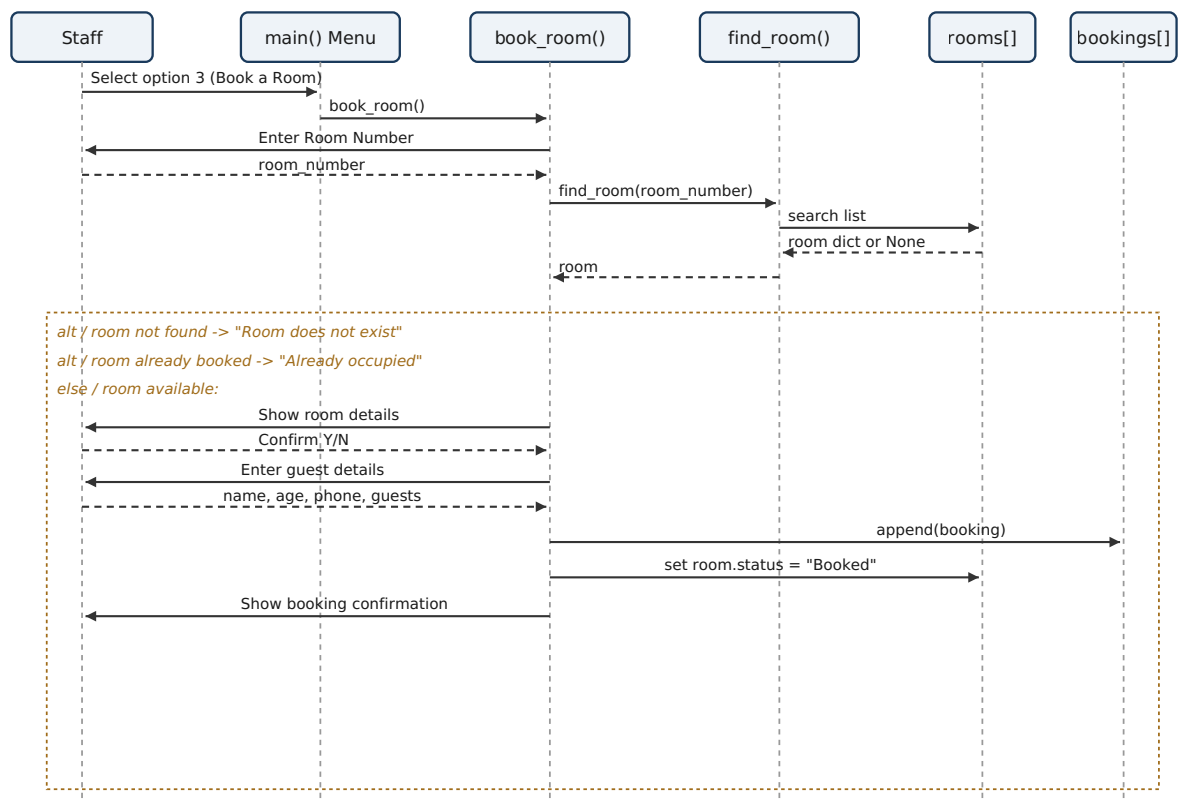


Figure 5: Sequence Diagram for the Booking flow

7.5 Storage / Schema Design

No external database is used; all data is held in memory using two lists of dictionaries for the duration of program execution.

rooms record		
Field	Type	Description
room	int	Unique room number (primary key)
type	str	Single / Double / Deluxe
price	int	Price per night (₹)
status	str	Available / Booked

bookings record		
Field	Type	Description
id	str	Unique Booking ID (primary key), e.g. B001
name	str	Guest name
age	int	Guest age
phone	str	Guest phone number
guests	int	Number of guests staying
room	int	Foreign key referencing rooms.room
type, price	str, int	Snapshot of room type/price at booking time (for stable billing/display)

Relationship: one room can have at most one active booking at a time, enforced in book_room() via the status check before a new booking is created.

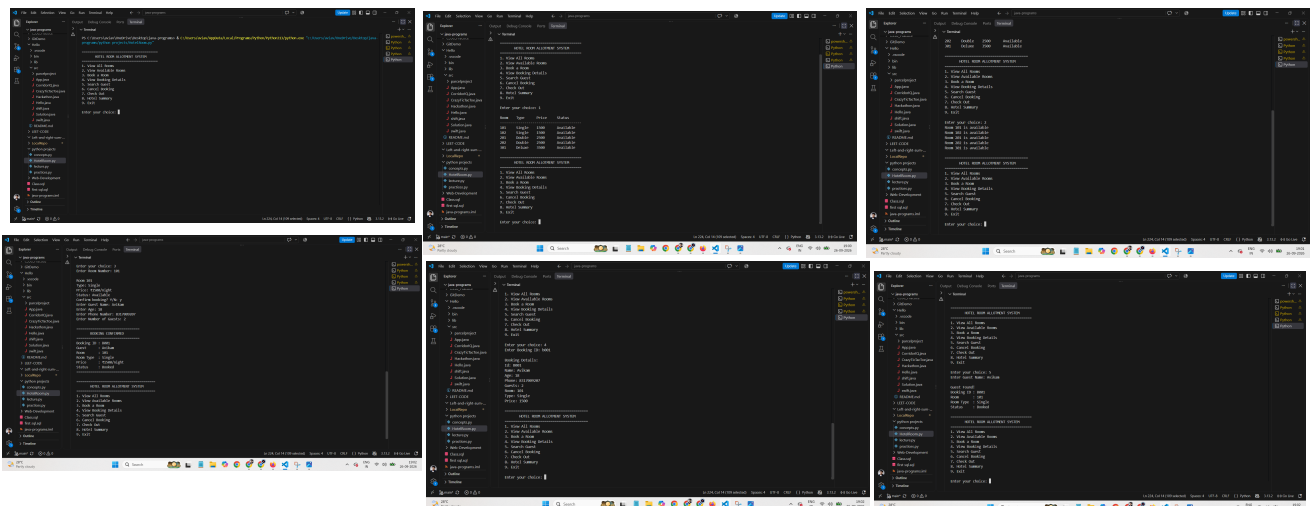
•

9. Implementation Details

The system is implemented in a single Python file, `hotel.py`, using only the standard library — no external dependencies are required. Key implementation points:

- **Data storage:** `rooms` and `bookings` are module-level lists of dictionaries, acting as an in-memory database for the program's runtime.
- **Lookup helpers:** `find_room(room_number)` and `find_booking(booking_id)` centralize the linear-search logic used by multiple features, avoiding duplicated loops.
- **Booking ID generation:** a module-level counter (`booking_counter`) is incremented and zero-padded (`str(n).zfill(3)`) to produce IDs like `B001`, `B002`.
- **Validation:** numeric inputs (room number, age, number of guests, nights) are parsed inside `try/except ValueError` blocks so malformed input produces a friendly error instead of a crash.
- **State transitions:** a room's `status` field is the single source of truth for availability, updated by `book_room()`, `cancel_booking()`, and `check_out()`.
- **Menu loop:** `main()` runs a `while True` loop reading a menu choice and dispatching to the relevant function via `if/elif`, exiting cleanly on option 9.

10. Screenshots / Results



14. Future Enhancements

Possible directions to extend this project beyond first-semester scope:

- Persist data between runs using a file (JSON/CSV) or a database (SQLite)
- Add a graphical interface (Tkinter) instead of a command-line menu
- Support multiple hotels or branches
- Add date-based bookings using Python's `datetime` module instead of a manual "number of nights" entry
- Add authentication for different staff roles (admin vs. front-desk)

15. References

- Python Software Foundation, *The Python Standard Library* — <https://docs.python.org/3/library/>
- Python Software Foundation, *Python Tutorial — Data Structures* — <https://docs.python.org/3/tutorial/datastructures.html>

]